

Intelligent Task Offloading in Vehicular Networks: A Deep Reinforcement Learning Perspective

Namory Fofana , *Member, IEEE*, Asma Ben Letaifa , *Member, IEEE*,
and Abderrezak Rachedi , *Senior Member, IEEE*

Abstract—The proliferation of connected vehicles and the Internet of Things (IoT) has made access to high-quality services increasingly viable. However, the growing number of vehicular applications poses challenges for embedded systems that need to perform tasks efficiently despite network fluctuations. To solve this problem, we have developed a Vehicular Edge Computing (VEC) system with a task offloading algorithm adapted to the Internet of Vehicles (IoV). Our solution uses a four-stage Stackelberg game and a reinforcement learning model. The approach consists of analyzing the vehicle state in real time to determine computational requirements and cost functions, implementing communication methods and cost functions, and using multi-agent reinforcement learning to design an experience-based offloading strategy. Through simulations, our algorithm demonstrates a balanced optimization of time, expense, work vehicle utility and service vehicle utility, while improving the probability of task success under various constraints.

Index Terms—Deep reinforcement learning, game theory, compute offloading, IoV, VEC, MEC, multi-agent scenarios.

I. INTRODUCTION

IN ORDER to improve driving efficiency, comfort and safety [1], modern cars are equipped with an increasing number of sensors, wireless communication systems and processing devices. But these improvements require a lot of processing and storage power, which cars don't always have [2], [3]. It is therefore crucial to maximize the use of available resources using a variety of computing techniques, including mobile or stationary vehicle clouds [4] and centralized or edge cloud computing. Despite these approaches, challenges persist in the ever-changing automotive landscape. Although it requires careful planning, compute offloading is a promising way to improve application performance and quality of service.

In addition, significant processing power is required for the optimal operation of intelligent transportation systems (ITS), which include safety applications, autonomous vehicles [5], [6], [7] and infotainment services [8]. Conventional cloud-based

solutions are insufficient due to problems such as inconsistent connections, bandwidth restrictions and transmission delays. The use of multiple-access edge computing (MEC) servers in the roadside infrastructure is a viable substitute that can meet the needs of real-time applications. An essential component of the 5G network, the MEC supports services that require high reliability and low latency. The efficiency of MEC can be further enhanced by using the vehicular computing (VC) paradigm, which integrates both stationary and moving vehicles [9], [10], [11]. In addition, these computing resources are made available through vehicle-to-vehicle (V2V) [12] and vehicle-to-infrastructure (V2I) [13] communications, which use technologies such as cellular networks (e.g., LTE and NR) and dedicated short-range communications (DSRC) [14], [15], [16].

Artificial intelligence (AI) is being widely adopted in sectors such as Industry 4.0, the Internet of Things (IoT) and automotive networks. In the context of sixth-generation (6G) wireless networks, AI is expected to play an important, distributed role, improving network performance and decision-making processes in a cost-effective [17] way. For example, AI can improve intelligent transport systems (ITS) by predicting traffic flow [18], optimizing traffic management [19], predicting speeds [20], detecting traffic accidents [21], identifying driving [22], [23] anomalies and detecting driver distractions [24]. More specifically, reinforcement learning (RL) and deep reinforcement learning (DRL) algorithms have shown their potential for improving task offloading performance in vehicles, even in complex, large-scale environments [25]. These algorithms are more effective than traditional methods in meeting these challenges.

Achieving optimal, stable, long-term performance in vehicle networks is a challenge due to their complexity and rapid change. Traditional optimization methods often fail under these conditions. While RL algorithms such as Q-learning struggle with significant environmental variations in vehicle task offload scenarios, DRL offers a promising solution. Using neural networks as function approximators, DRL adapts to the complex environment of vehicle offloading and enables real-time decision-making through iterative learning [26]. This enables DRL agents to respond effectively to environmental changes without the need for a pre-existing model, relying instead on accumulated experience to refine decision policies. In our study, we use DRL as an optimization tool to optimally allocate computing resources, maximizing system utility while minimizing task processing times and energy consumption.

Manuscript received 20 October 2023; revised 24 May 2024; accepted 8 July 2024. Date of publication 15 July 2024; date of current version 16 January 2025. The review of this article was coordinated by Dr. Cailian Chen. (*Corresponding author: Abderrezak Rachedi.*)

Namory Fofana and Asma Ben Letaifa are with the Mediatron Research Lab Sup'Com, University of Carthage, Tunis 2083, Tunisia (e-mail: namoray.fofana@supcom.tn; asma.benletaifa@supcom.tn).

Abderrezak Rachedi is with Computer Science Lab, Université Gustave Eiffel (CNRS), 77420 Marne-la-Vallée, France (e-mail: abderrezak.rachedi@univ-eiffel.fr).

Digital Object Identifier 10.1109/TVT.2024.3427814

However, despite these advances, challenges remain in ensuring the convergence of reinforcement learning algorithms in highly dynamic environments such as vehicle networks. Slow convergence can affect operational efficiency and limit the real-time adaptability of DRL agents [27]. To remedy this, integrating game-theoretic principles into DRL algorithms is a promising approach. Game theory provides a robust framework for analyzing interactions between interested participants, making it well suited to the dynamic behavior of mobile devices. It discourages unilateral deviations by participants by identifying the best policies under current conditions. Recent research has applied game theory to compute offloading decisions in the MEC (Mobile Edge Computing) environment, enabling end-users to make decentralized offloading decisions autonomously [28]. This simplifies optimization by allowing decisions to be based on local observations rather than global information. The distributed nature of game theory aligns well with multi-user offloading scenarios, aiming to achieve Nash equilibrium.

By combining game-theoretic strategies with DRL, we can enhance the adaptation speed of DRL agents to environmental changes, improving overall system performance. This hybrid approach offers a promising research direction for overcoming convergence challenges in vehicular networks and opens up new possibilities for delay-sensitive applications.

In our study, we developed a game-theoretic model and Deep Reinforcement Learning Framework to analyze cooperative relationships between vehicles and RSU and define the utility function for offloading tasks, leading to optimal task offloading decisions involving task allocation and resource coordination.

Thus, our contributions are as follows:

- 1) We integrate Mobile Edge Computing (MEC) technology into Vehicular Ad-hoc Networks (VANET) to establish a Vehicular Edge Computing (VEC) system. This innovative approach utilizes vehicles to provide efficient and rapid computing services, overcoming the limitations associated with fixed infrastructures;
- 2) We introduce a novel task offloading algorithm utilizing the Stackelberg game framework and deep reinforcement learning. Our objective is to reach a balance between minimizing time delays and optimizing task costs. This approach leverages strategic decision-making, enhancing the efficiency of task-offloading processes;
- 3) We conduct experiments to find the Nash equilibrium between service agents and task vehicles;
- 4) We compare the performance of our proposed algorithm with other algorithms and find that it achieves higher system utility and better overall performance.

The rest of this paper is structured as follows: Section II reviews related work on computing offloading optimization strategies and their limitations. Section III introduces the VEC system, system models, and problem formulation. In Section IV, we present our proposed solution and describe the algorithmic model used to implement the task offloading strategy. Section V analyzes the simulation results of our scenario. Finally, Section VI concludes the paper.

II. RELATED WORK

Task offloading in vehicle networks involves transmitting a task to devices or infrastructure with more computing resources, executing the task there, and receiving the results back. There are three main strategies for task offloading. In the local execution strategy, the entire calculation process is done within the vehicle itself. In the full task offload strategy, the entire task is processed by external nodes like MEC servers or other vehicles. In the partial offloading strategy, only a portion of the task is processed locally, while the rest is offloaded to other vehicles or MEC servers. One major challenge in task offloading is making the optimal decision on whether to offload a job, choosing the best destination for offloading, and determining which part of the job should be offloaded. These decisions aim to improve the overall efficiency of the compute offloading process.

In this article, we reviewed various task offloading approaches from the perspectives of machine learning and game theory.

A. From Machine Learning Perspective

Machine learning, particularly reinforcement learning (RL), has become a pivotal approach in optimizing complex systems by enabling adaptive and intelligent decision-making processes. In the context of mobile offloading, RL techniques have been employed to address dynamic and uncertain environments, enhancing both efficiency and security. Several studies have leveraged machine learning to devise strategies that optimize task offloading, resource allocation, and manage computational loads effectively. Xiao et al. [29] introduced a reinforcement learning (RL)-based strategy for mobile offloading in the presence of intelligent attacks. In their approach, tasks are offloaded by the user to an access point (AP), while an intelligent attacker attempts to disrupt the transmission. To mitigate this interference, the user must carefully select the offload rate. The environment is characterized by dynamic factors such as unknown attack costs, channel gains, and detection accuracies. The problem is framed as a secure mobile offloading game and addressed using Q-learning. Experimental results demonstrated that Q-learning improves both offloading rate and security by 86% and 6%, respectively, compared to a random offloading scheme. However, the scalability of this solution is hampered by the rapid increase in temporal and spatial complexities as the system state size grows, limiting its applicability to larger state spaces.

Similarly, Chen et al. [30] developed a dynamic computational load shedding strategy using deep reinforcement learning (DRL) in a Mobile Edge Computing (MEC) network. The goal was to reduce energy consumption while ensuring timely task execution. The MEC network consisted of a base station with an attached server and multiple mobile users, with tasks arriving randomly and channel conditions varying over time. The system determined power allocation for local execution and load shedding in each time slot. To efficiently manage the power allocation, the researchers employed the Deep Deterministic Policy Gradient (DDPG) method. This method allowed the system to learn effective load shedding policies based on user

states, such as buffer length and channel state, and adaptively allocate power for local execution and task offloading. Numerical results indicated that the DDPG-based policy outperformed the discrete Q-learning (DQN) based policy in terms of reducing computational costs.

In addition, Tan et al. [31] explored user mobility in vehicular networks and the challenges posed by dynamic changes in downlink channel conditions, computational capabilities, caching states, and vehicle mobility. To address these challenges, they proposed a deep Q-learning framework that considers environmental dynamics. The framework utilizes system states such as available roadside units (RSUs), mobile edge computing (MEC) servers, vehicles, caches, and the number of contacts. At each time interval, the system decides which RSUs or vehicles to assign to requesting vehicles, whether to cache requested content, and whether to offload computational tasks. The goal is to minimize communication, storage, and calculation costs. However, a limitation of this framework is its centralized nature, which requires control over all vehicles in the network. This centralized control can lead to exponential computational complexity as the network expands.

In conclusion, RL-based offloading approaches designed with Q-learning and Deep Q-Network algorithms may not perform well in complex MEC offloading environments. RL-based methods require significant time for the agent to explore and learn the policy for higher cumulative rewards, especially as the dimensions of states and actions increase, making the exploration and learning process time-intensive.

B. From Game Theoretic Perspective

Game-based theories provide a comprehensive set of mathematical tools for modeling and analyzing interactions related to the behaviors of individual or group users in computing environments [32]. Game theory [33] is particularly suitable for modeling and examining resource allocation problems in various computing contexts, significantly simplifying complex problems.

The authors of the article [34] propose a method for resource allocation in cloud-edge computing networks using data offloading and task allocation. The master device, which has tasks to perform, can choose to process them locally on nearby slave devices or in the cloud instead of sending them to a remote server. In their optimization problem, the authors consider factors such as communication cost and computation cost. They formulate the problem as a two-stage Stackelberg game, where the slave devices first propose the amount of data execution units they can provide, and then the master device defines the pricing strategies for each slave device. This approach efficiently optimizes resource allocation by finding the best combination of local and remote processing based on cost factors.

Furthermore, game theory is a powerful tool for analyzing the interactions between multiple independent and selfish entities that must collaborate to achieve their goals. Recently, various game-theoretic incentive mechanisms have been proposed to encourage compute servers (CS) or edge servers (ES) to

participate in the offloading process. Zeng et al. [35] analyzed the interaction between a requesting vehicle and edge servers using the Stackelberg game, identifying the optimal strategy for both parties. Similarly, Li et al. [36] proposed a computational offloading mechanism based on a two-stage Stackelberg game to examine interactions between multiple edge clouds and multiple Industrial Internet of Things (IIoT) devices, introducing two dynamic iterative algorithms to find the Stackelberg equilibrium.

Zeng et al. [37] developed a reputation-based incentive mechanism using the Stackelberg game for software-defined vehicles, analyzing optimal strategies for both parties involved. Additionally, Chang and Wei [38] proposed a uniform-price access control mechanism for ESs using the Stackelberg game to study the relationship between CSs and ESs. Moreover, Hong et al. [37] focused on the multi-hop computational offloading problem in the IIoT-edge-cloud computing model, using game theory to minimize computation time and energy consumption for each task. Cao and Cai [39] formulated the multi-user computational offloading problem for cloudlet-based mobile cloud computing as a non-cooperative game, proposing a fully distributed computational offloading algorithm to maximize the number of beneficial mobile cloud computing devices. These studies highlight the use of game theory in addressing various computational offloading problems and improving efficiency in different scenarios of mobile computing.

In another important contribution, Zhang et al. [40] conducted a study on a drone-assisted multi-access edge computing system, introducing a game-theoretic method to minimize the weighted cost of delay and energy consumption. In addition, another study [41] proposed a joint optimization framework combining the Stackelberg game and a matching game. This framework models the pricing problem using the Stackelberg game and analyzes the matching problem using the matching game. Unlike traditional cloud-edge collaboration scenarios, this framework focuses on a scenario in which computational tasks are offloaded from the cloud server (CS) to edge servers (ES), with the CS playing the role of leader and the ES that of follower. The study also examines the impact of task execution time and ES satisfaction on CS and ES efficiency in this scenario.

Other researchers have explored various aspects of game theory in computing environments. For example, they have examined power control in single MEC systems [42], task offloading to cloudlets [43], and vehicle fog computing with user priority requests [44]. They have also studied non-cooperative games based on forward-looking theory [45], joint MEC and D2D offloading systems [46], and cost minimization in single MEC servers [47]. Each study demonstrates the usefulness of game theory for optimizing resource allocation and offloading decisions.

In conclusion, although game theory can guarantee Nash equilibrium, it does not always lead to the globally optimal solution. In multi-agent environments, despite the increased flexibility it offers end-users and its ability to decentralize IT offloading decisions, it does not always guarantee the best result for all system participants.

C. From Combination of RL and Game Theoretic

In a multi-SD MEC system, finding the optimal offloading policy can be achieved through multi-agent reinforcement learning (MARL) and game theory [48]. MARL allows each agent to interact with the environment, receive rewards, and improve its policy over time. However, incorporating multiple agents can lead to challenges such as larger system size, increased complexity, instability, and slower learning speeds. To address these challenges, combining MARL with game theory is a logical approach. This combination, referred to as MARL, leverages the cooperative or competitive relationship between agents to optimize the offloading policy. By utilizing game theory, MARL can overcome the limitations of MARL in a multi-agent environment and provide a globally optimal strategy.

Some existing studies have combined RL and game theory to solve offload decision-making problems in the MEC environment. For example, in [49], the problem of server activation in MECs was formulated as a minority game, and RL was used to solve the Nash equilibrium of non-cooperative games. In addition, [50] examined security issues in offloading decisions based on RL and game theory, deriving optimal offloading rates and reducing the attack rate of smart attackers on IoT devices and In [51], Zhan et al. investigated the computational offloading game under conditions of incomplete information sharing, employing the Proximal Policy Optimization (PPO) algorithm.

In addition, the main challenge of optimal resource allocation, such as task offloading using game-theoretic models, is to achieve Nash equilibrium. The most common solution is to use the Stackelberg game framework. In other words, the most appropriate game-theoretic model for resource allocation and computational offloading is the Stackelberg game model. [52]

In our paper, we will use the combination of MARL and Stackelberg game theory to address problems related to task allocation and service pricing.

III. SYSTEM DEFINITION

In this section, we begin by outlining the fundamental elements of our VEC system. Subsequently, we introduce the communication and computation model, followed by the formulation of the offloading decision problem.

A. VEC System

As shown in Fig. 1, this article considers the computation offloading of computationally-intensive tasks which cannot be performed within a limited timeframe by the vehicle's on-board resources alone in the VEC network. The high-speed movement of vehicles leads to rapid changes in network topology, resulting in unstable communication and high information exchange overhead. To optimize the data transmission between vehicles and improve the QoS of the VEC system, in this research, we use a straight road with dimensions L and W to define the length and width of the road. The network comprises two roadside units (RSUs) located on either side of the road [53], equipped with extensive computing resources and interconnected by fiber optics, with one RSU connected to the cloud server to save on

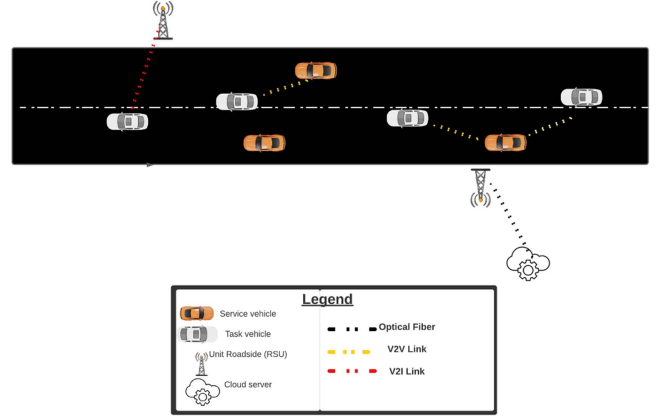


Fig. 1. Task offloading system model.

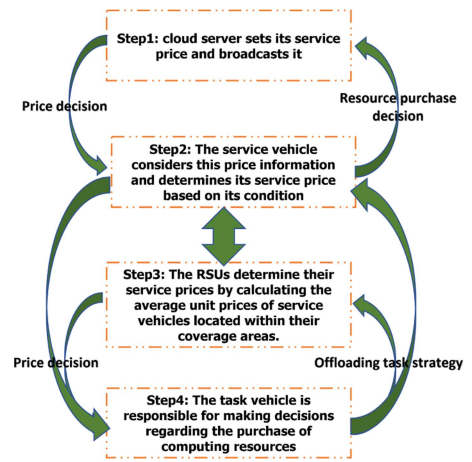


Fig. 2. Our four-step Stackelberg game model.

construction cost [31]. Within each time slot, inactive vehicles in the system possess a probability of generating computational tasks. Vehicles harboring these tasks are denoted as “Task Vehicles (TV),” whereas vehicles with idle computing resources are referred to as “Service Vehicles.” [54], [55], [56], [57], [58], [59]. TV have the flexibility to adopt different strategies when purchasing computing resources, based on their battery level, task priority, and associated costs. RSUs and SVs offer different levels of service, depending on their workload and available computing resources. Service agents charge according to their status and the status of the task during processing.

V2V communication is utilized when the distance between a SV and TV is less than the V2V range of 300 meters. Otherwise, V2I communication is utilized.

Our paper proposes a Stackelberg game model that consists of four stages, as shown in Fig. 2.

- In the first step, the cloud server broadcasts the unit price of its computing power to the service vehicle based on its maximum computing power.

- In the second step, the SV determines the unit price of its computing power by considering its current CPU utilization rate and the unit price of computing power from the cloud server and then broadcasts this unit price to other service vehicles and surrounding RSUs.

- Moving to the third step, RSUs covering an area with service and TV collect the different unit prices of SV, average them, and communicate the result as service prices to TV during V2I communication.

-Finally, in the fourth step, the TV selects offload strategies based on several factors, including price sensitivity, delay sensitivity, task priority, task success sensitivity, and the task and offload mode used.

This approach allows for efficient coordination between the different actors involved in the VEC network, providing task vehicles with a better understanding of the computational resource prices and offload options available. In addition, this strategy can help maximize user satisfaction by taking into account various factors such as cost and quality of service.

B. Communication Model

1) *Inter-Vehicular Communication (V2V)*: V2V communication employs Orthogonal Frequency Division Multiplexing (OFDM) [60] technology, which enables vehicles to transmit without interference due to its high resistance to electromagnetic interference and more efficient utilization of the available bandwidth. The path loss ϕ of the V2V communication at time t is defined as (1) [61]:

$$\Phi_{V2V}(t) = 63.3 + 17.7 \log_{10}(d_{(p,q)}(t)) \quad (1)$$

where $d_{p,q}(t)$ denotes the distance between vehicles p and q at time t .

In our scenario, we assume that the Additive White Gaussian Noise (AWGN) value is 0 and that certain variables, including the communication bandwidth B , the transmission channel fading factor h , the Gaussian white noise power N , and the transmission power of the vehicular communication equipment P , are known. Using Shannon's equation, we can then calculate the data transfer rate between two communicating vehicles, denoted as p and q at time t is defined as (2) [62]:

$$Dr_{V2V} = B_{V2V} \log_{10} \left(1 + \frac{P_{V2V} 10^{\frac{\Phi_{V2V}(t)}{10}} h^2}{N} \right) \quad (2)$$

2) *Communication Model in V2I Mode*: In V2I communication mode, we denote by ϕ_{V2I} the path loss [63] between the vehicle p and the RSU r , which is defined as (3):

$$\phi_{V2I} = d^{-\delta} \quad (3)$$

where d denotes the distance between the vehicle and the RSU, and δ denotes the path loss factor. We model the downstream and upstream links between vehicles and RSUs using Rayleigh flat-fading channels. According to Shannon's equation, the data transmission rate between the vehicle p and the RSU r is defined as (4):

$$Dr_{V2I} = B_{V2I} \log_{10} \left(1 + \frac{P_{V2I} \phi_{V2I} h^2}{N} \right) \quad (4)$$

C. Cost Service Model

According to [65], the task of vehicle p can be represented by tuples $TV_p = \{I_p, L_p, \phi_p, t_p^{\max}\}$, $p \in N$, where I_p represents

the size of the task data for computation, L_p denotes the number of cycles required by the CPU to complete the calculation of one bit of data., ϕ_p is the size of the output data, $t_p^{\max}$ is the maximum latency of the task and N is the total number of vehicles that need to offload task.

This modeling approach allows us to better understand the requirements and characteristics of the tasks being offloaded, which can help in making more informed decisions about which resources to utilize for offloading. By considering the size, computational requirements, completion time, and output of the offloaded task, we can optimize the selection of appropriate RSUs or service vehicles to ensure efficient and effective task offloading.

1) *Cost Setting of Service Vehicles*: In the Vehicular Edge Computing (VEC) system, the SV adapts its pricing strategy to match the specific demands of each task. It initiates by broadcasting its base price P_q^{base} and then computes a differential price proportional to the task's priority. This approach ensures load balancing across all edge servers by raising the unit price of computing power for high-load servers and lowering it for low-load servers. To calculate the unit price of computing power charged to the TV, Equation (5) is used.

$$P_{q,p} = P_q^{base} + w_a \frac{I_p}{t_p^{\max}} + w_b \left(\frac{f_q^{free}}{f_q^{\max}} - 1 \right) \quad (5)$$

In this equation, P_q^{base} is the base price set by the service vehicle v for computing power. It is likely to cover the basic cost of providing computing services, such as infrastructure maintenance and operational expenses.

Then, we have the normalization factors (w_a and w_b): where w_a is the price normalization factor and w_b is the server utilization normalization factor. These factors are used to scale the impact of differential price and server utilization on the overall unit price. They enable adjustments to be made according to the relative importance of these factors in the pricing model.

We have also $w_a \frac{I_p}{t_p^{\max}}$ the differential price according to the priority of each task. It reflects the variation in the price based on the importance or urgency of the task. This ensures that tasks with higher priority are completed more quickly, but potentially at a higher cost.

Finally, we have $w_b \left(\frac{f_q^{free}}{f_q^{\max}} - 1 \right)$, the server CPU utilization rate which reflects the current use of server CPU capacity. It adjusts the unit price according to the server's workload, ensuring that high-load servers are charged more for computing power, while low-load servers are charged less. Here, f_q^{free} signifies the available or unused computing resources for a specific service vehicle q at a given time, and $f_q^{\max}$ represents the maximum computing resources that can be allocated to a service vehicle within the system.

Through the application of this pricing model, service vehicles can dynamically adjust their prices, ensuring competitive rates while maximizing the efficient utilization of all edge servers.

2) *Cost Setting of RSU*: The proposed strategy aims to enable task vehicles to make informed decisions when selecting an offloading strategy by taking into account the base price, P_r^{base} ,

calculated by the service RSU r . This base price is distributed to task vehicles within the coverage area of the RSU and is adjusted when the communication mode switches from V2V to V2I based on task priority. Equation (6) is used to determine the RSU's base price by taking the weighted average of the base prices of all service vehicles within the coverage area:

$$P_r^{base} = \frac{\sum_{i=1}^m P_i^{base} \cdot W_i}{\sum_{i=1}^m W_i} \quad (6)$$

Where

$$W_i = \frac{f_i^{free}}{\sum_{i=1}^m f_i^{free}} \quad (7)$$

P_i^{base} is the base price for surrounding service vehicles. Each service vehicle in the RSU coverage area offers computing services at a certain base price. This price probably reflects factors such as the vehicle's computing capacity, operating costs and profit margin. The base price serves as the starting point for calculating the RSU base price.

W_i is the weight assigned to each service vehicle i . It is determined by comparing its computational capacity (represented by f_i^{free} , the free computational resources available) with the average computational capacity of all surrounding service vehicles in the coverage of the RSU. This weight represents the relative contribution of each service vehicle's computational capacity to the overall pool available within the RSU's coverage area.

$\sum_{i=1}^m P_i^{base} \cdot W_i$ is the weighted price which is the sum of the products of the base price and the corresponding weight for each service vehicle within the coverage area of RSU r . It serves as a normalization factor and represents the total contribution of each service vehicle to the RSU's base price, weighted by their computational capacity. This approach considers the system's dynamics in terms of base pricing.

Equation (8) computes the unit price of computing power, denoted as $P_{r,p}$, charged to the TV p by the RSU r . This price is calculated by taking various factors into consideration, including:

P_r^{base} , the base price set by the service RSU u for computing power, $w_a \frac{I_p}{t_{p,max}}$ the differential price according to the priority of each task and $w_b \left(\frac{f_r^{free}}{f_{r,max}} - 1 \right)$, the server CPU utilization rate which reflects the current use of server CPU capacity.

$$P_{r,p} = P_r^{base} + w_a \frac{I_p}{t_{p,max}} + w_b \left(\frac{f_r^{free}}{f_{r,max}} - 1 \right) \quad (8)$$

This approach enables TVs to select the most cost-effective offloading strategy based on the unit price of computing power charged by the RSU, which can ultimately reduce the overall cost of the system.

D. Task Compute Model

In this proposed model, there are a total of M service vehicles, N work vehicles, and 2 Road Side Units (RSUs). The task vehicle, referred to as TV_p , initiates the offloading process by sending offloading requests to all surrounding service vehicles and RSUs. The purchase strategy for TV_p is determined by f_i ,

which can be either $f_p^{(1)} = (f_{p,0}, \dots, f_{p,q})$ with $(0 < q \leq M)$ or $f_p^{(2)} = (f_{p,0}, f_{p,r})$ with $(r \in \{1, 2\})$.

To efficiently offload task TV_p , it is divided into $v + 1$ sub-tasks denoted as $\{t_{p,0}, \dots, t_{p,q}\}$ where $(0 < q \leq M)$, or 2 sub-tasks represented as $\{t_{p,0}, t_{i,r}\}$ where $(r \in \{1, 2\})$, depending on whether V2V or V2I communication is utilized. Sub-task $t_{p,0}$ is computed locally, while sub-tasks $T_{p,q}$ are offloaded to corresponding Service Vehicles and $t_{p,r}$ to Roadside Units. The decision to use V2V or V2I communication for offloading depends on the strategies V2V only, V2I only, or V2V2I [64], where V2V2I determines the offloading of tasks to the cloud server.

1) *Local Task Compute*: Considering the variety of vehicles, it's acknowledged that their computing resources might not be ample to manage the entire task without risking resource exhaustion. Therefore, the TV_p has a maximum resource f_p^{max} , and $f_{p,0}^{(0)}$ represents the resource allocated by task vehicle i for computing its local task $T_{p,0}$, with the constraint $f_{p,0}^{(0)} \leq f_p^{max}$.

2) *Service Vehicles Task Compute*: We consider vehicles as mini-servers providing task-processing services. When a SV q receives requests from p TVs, (where $0 < p \leq N$), to offload tasks within a specific time interval, it communicates with all p TVs if the total computing resource requirements f_q is within the available resource f_q^{free} .

If the total computing resource requirements surpass the maximum capacity, the surplus is computed as $f_{q,cloud} = f_q - f_q^{free} = \sum_{q=1}^n f_{q,p,cloud}$. In this case, the service vehicle procures extra resources from the cloud server, denoted as $f_{q,p,cloud}$.

When a task vehicle receives an offload message [65], it transmits data size information $(\frac{f_{p,q} - f_{p,q,cloud}}{L_p})$ to the service vehicle and size data $(\frac{f_{p,q,cloud}}{L_p})$ to the cloud server. Here, L_p represents the number of cycles required by the CPU to process a single bit of information.

3) *RSU Task Compute*: Regarding the RSUs, in V2I mode, they can receive offload requests from a maximum of N TVs within a specific time interval. Consequently, they broadcast messages to all task vehicles if the total requested computing resources, denoted as $f_r = \sum_{r=1}^n f_{r,p}$ is within the available resource f_r^{free} .

When the cumulative query computing resources surpass the maximum available capacity, the RSU will utilize the cloud server's computing resources to manage the surplus. In this scenario, the computing resources utilized from the cloud server are precisely equal to $f_{r,cloud} = f_r - f_r^{free} = \sum_{r=1}^n f_{r,p,cloud}$ and the total computing resources drawn from the cloud server are denoted as $f_{r,p,cloud}$.

Upon receiving the offload message, the task vehicle transmits data of size $(\frac{f_{p,r} - f_{p,r,cloud}}{L_p})$ to the nearby RSU and data of size $(\frac{f_{p,r,cloud}}{L_p})$ to the cloud server.

Once the computation is finished and the task is completed, all the TVs receive a notification.

E. Task Completion Time

In our scenario, a task vehicle i is responsible for computing the first subtask $t_{p,0}$ locally. Additionally, it has the flexibility

to choose between V2V and V2I communication modes and the decision to offload a subtask $T_{p, \cdot}$ to a service agent, which can be either a service vehicle q or a service RSU r . In this process, the task vehicle needs to procure computing resources $f_{p, \cdot}$ from the selected service agents, and these resources are subject to constraints as the service agent possesses a limited amount of available resources, denoted as f_q^{free} in the service vehicle or f_r^{free} in the RSU.

If the necessary resources are within the available capacity, the service agent can opt for either V2V or V2I mode without needing to acquire extra computing resources from the cloud server. In this scenario, the task completion time involves three distinct phases:

- 1- sending time t_p^{send}
- 2- execution time t_p^{exec}
- 3- and return time t_p^{return} .

1) *Task Completion Time in Local*: When the TV_p processes its first subtask $t_{p,0}$, the delay is defined as:

$$t_p^{(0)} = \frac{L_p I_p}{f_p^{local}} \quad (9)$$

Where f_p^{local} is the local computing resource to process the task $t_{p,0}$.

2) *Task Completion Time When Computing Resources are Sufficient*: The processing time of a subtask TV_p in V2V mode can be computed using a formula similar to equation (10).

$$t_{p,q}^{(1)} = t_{p,q}^{send} + t_{p,q}^{exec} + t_{p,q}^{return} \quad (10)$$

With $t_{p,q}^{send} = \frac{a I_p}{R_{V2V}}$, $t_{p,q}^{exec} = \frac{a L_p I_p}{f_{p,q}}$, and $t_{p,q}^{return} = \frac{a \phi_p I_p}{R_{V2V}}$

- In V2I compute mode, the processing time is:

$$t_{p,r}^{(1)} = t_{p,r}^{send} + t_{p,r}^{exec} + t_{p,r}^{return},$$

$$\text{With } t_{p,r}^{send} = \frac{a I_p}{R_{V2I}}, t_{p,r}^{comp} = \frac{a L_p I_p}{f_{p,r}}, \text{ and } t_{p,r}^{return} = \frac{a \phi_p I_p}{R_{V2I}}$$

We have also $a = \frac{f_{p, \cdot}}{f_{total}^p}$ indicates the proportion of task offloading, f_p^{total} indicates the total computing power used during the offloading process, f_p^{local} denotes the computing power of the task vehicle, R_{V2V} [60] and R_{V2I} [67] denotes the data transfer rate.

The duration needed to complete the task p is denoted by equation (11):

$$t_p^{complete} = \max\{t_p^{(0)}, t_p^{(1)}\} \quad (11)$$

Where $t_p^{(1)}$ is the execution time of offloaded subtasks $T_{p, \cdot}$ to service agents, either service vehicles q or a service RSU r in the neighborhood of TV_p .

3) *When Service Computing Resources are Insufficient*: When the computing resources in service vehicles and RSUs are insufficient, cloud servers are employed. If the computing resources purchased from service agents surpass the available resources of the service vehicle, subtasks $t_{p,1} \sim T_{p,q}$ with $(0 < q \leq M)$ is divided into $\{t_{p, \cdot}^{(1)}, t_{p, \cdot}^{(2)}\}$.

In this scenario, the task time is divided into three segments: the local execution time $t_p^{(0)}$, the processing time by service agents $t_{p, \cdot}^{(1)}$, and the processing time by cloud servers $t_{p, \cdot}^{(2)}$.

The local processing time and the execution time of the service agents have been previously defined in equations (9) and (10). The processing time of the cloud server utilizing V2I communication can be calculated using the following equations.

- **If the service provider is a Service Vehicle (SV)**

The duration for processing on the cloud server is:

$$t_{p,q}^{(2)} = t_{p,q,r}^{send} + t_{p,q,r}^{rsu,cloud} + t_{p,q,r}^{exec,cloud} + t_{p,q,r}^{cloud,rsu} + t_{p,q,r}^{return} \quad (12)$$

The time taken for data sending from the task vehicle to the neighboring relay RSU is: $t_{p,q,r}^{send} = \frac{a I_p (f_{p,q} - f_q^{free})}{R_{V2I} f_{p,q}}$.

The Relay RSU upload time to the cloud server is:

$$t_{p,q,r}^{rsu,cloud} = \frac{a I_p (f_{p,q} - f_q^{free})}{R_{LAN} f_{p,q}} + c_k \Delta t, c_k \in \{0, 1\}.$$

In our proposed model, R_{LAN} is set to 10^9 [63] Gb Ethernet. Due to the short distance between two RSUs, transfer time is treated as a constant denoted by Δt , where c_k represents the number of hops needed to travel between two RSUs.

The execution time on the cloud server is calculated as: $t_{p,q,r}^{exec,cloud} = \frac{a L_p I_p}{f_{p,q} - f_q^{free}}$.

The transfer time from the cloud server to the relay RSUs is:

$$t_{p,q,r}^{cloud,rsu} = \frac{a \phi_p I_p (f_{p,q} - f_q^{free})}{R_{V2I} f_{p,q}} + c_k \Delta t, c_k \in \{0, 1\}.$$

The task result return time is: $t_{p,q,r}^{return} = \frac{a \phi_p I_p (f_{p,q} - f_q^{free})}{R_{V2I} f_{p,q}}$.

- **If the service provider is a RSU**

The duration for processing on the cloud server is defined as:

$$t_{p,r}^{(2)} = t_{p,r,r'}^{rsu,cloud} + t_{p,r,r'}^{exec,cloud} + t_{p,r,r'}^{cloud,rsu} + t_{p,r,r'}^{return} \quad (13)$$

where r and r' represent the initial and final Relay RSUs that provide coverage to TV_p in the process of computing the task offloaded.

The transfer time from the RSU to the cloud server is:

$$t_{p,r,r'}^{rsu,cloud} = \frac{a I_p (f_{p,r} - f_r^{free})}{R_{LAN} f_{p,r}} + c_{r'} \Delta t, c_{r'} \in \{0, 1\},$$

Where $c_{r'}$ indicates the number of hops required to travel between two RSUs.

The execution time on the cloud server is defined as:

$$t_{p,r,r'}^{exec,cloud} = \frac{a L_p I_p}{f_{p,r} - f_r^{free}}.$$

The transfer time from the cloud server to the relay RSU is:

$$t_{p,r,r'}^{cloud,rsu} = \frac{a \phi_p I_p (f_{p,r} - f_r^{free})}{R_{V2I} f_{p,r}} + c_{r'} \Delta t, c_{r'} \in \{0, 1\}.$$

The task result return time is: $t_{p,r,r'}^{return} = \frac{a \phi_p I_p (f_{p,r} - f_r^{free})}{R_{V2I} f_{p,r}}$.

Thus, the task completion time of the task vehicle can be defined as equation (14):

$$t_p^{complete} = \max\{t_p^{(0)}, t_p^{(1)}, t_p^{(2)}\}. \quad (14)$$

F. Problem Formulation

In our offloading scenario, the decision-making process is vital for both task vehicles and service agents. To facilitate this process, we use utility functions that take various factors into account, guaranteeing a balanced approach in terms of time, costs, success rate, etc.

1) *From Task Vehicle Perspective:* In our study of task vehicle offloading strategies, the TV utility function, as defined by equation (15), occupies a central position in optimizing task time, controlling task-related costs and ensuring task success. Task vehicles navigate between V2V (vehicle-to-vehicle) or V2I (vehicle-to-infrastructure) offloading options, taking into account factors such as task priority, computational requirements, cost considerations and network conditions.

The objective of each task vehicle is to maximize its utility by purchasing the least amount of computing resources possible, while achieving the most successful task offload.

$$\max F_{\text{task}} = xF_{\text{time}} - yF_{\text{pay}} + zF_{\text{success}}, \quad (15)$$

where $0 < x \leq 1, 0 < y \leq 1, 0 < z \leq 1$ and $x + y + z = 1$.

In the given equation, the variables have specific meanings in the context of our study: x symbolizes the price sensitivity factor, y represents the time sensitivity factor, and z represents the task success factor. The term F_{time} refers to the satisfaction derived from time savings, which increases in proportion to the computational capacity of the task vehicle. Conversely, F_{txtpay} refers to the satisfaction associated with cost differentials, which decreases as task expenditure increases.

The satisfaction F_{time} can be defined as:

$$F_{\text{time}} = \gamma_1 P_r \ln(1 + t_{\text{save}}), \quad (16)$$

$\gamma_1 > 0, P_r > 0$, where γ_1 represents a scaling factor applied to the satisfaction metric. Its positive value ensures that the satisfaction metric increases with the time saved by offloading a task. A higher value of γ_1 amplifies the impact of time savings on overall satisfaction, emphasizing the importance of task offloading efficiency. P_r represents the maximal resource required to compute a task by the task vehicle. It provides a reference point for evaluating the efficiency of task offloading. Tasks with resource requirements closer to P_r are considered more computationally intensive, and hence, the potential time savings from offloading such tasks are more significant and the logarithmic function $(1 + t_{\text{save}})$ ensures that the satisfaction measure increases non-linearly with time savings. This reflects the decrease in satisfaction performance as time saved increases, reflecting the idea that small time-saving improvements are more useful for tasks with shorter execution times.

The calculation of t_{save} is defined as:

$$t_{\text{save}} = t_p^{(3)} - \max\{t_p^{(0)}, t_{p,\cdot}^{(1)}, \dots, t_{p,\cdot}^{(2)}\}, \quad (17)$$

$$t_{\text{save}} = \frac{aI}{f_{\text{local}}} - \max\left\{\frac{aI}{f_{\text{total}}}, t_{p,\cdot}^{(1)}, \dots, t_{p,\cdot}^{(2)}\right\}, \quad (18)$$

Here, t_{save} signifies the temporal advantage gained through the strategic offloading of tasks utilizing either V2V or V2I communication. The choice of the offloading mode profoundly

influences the task completion duration. Specifically, $t_p^{(0)}$ represents the time taken for local task completion when the initial subtask $t_{p,0}$ is processed within the vehicle itself. $t_{p,\cdot}^{(1)}$ signifies the completion time when all service agents suffice for the task. $t_{p,\cdot}^{(2)}$ reflects the time taken when cloud server resources are utilized, and $t_p^{(3)}$ denotes task completion solely through local computation.

Furthermore, the maximal resource allocation for a given task, denoted as P_r is defined as:

$$P_r = \frac{I}{t_{\text{max}}}, \quad (19)$$

where I represents the size of the task TV_p , and t_{max} signifies the maximum time required for computing task TV_p without dividing it into subtasks.

F_{success} represents the reward associated with the task, denoted as a positive constant (G). This reward of a task depends on its timely completion.;

-if the task is completed on time, the reward remains G .

- However, if the task exceeds the scheduled time, the reward becomes negative, equal to $-G$.

Mathematically, this can be defined as:

$$F_{\text{success}} = F_{\text{success}}^{V2V} + F_{\text{success}}^{V2I}, \quad (20)$$

With $F_{\text{success}}^{V2V} > 0, F_{\text{success}}^{V2I} > 0$, where F_{success}^{V2V} and F_{success}^{V2I} correspond to the success of V2V and V2I offloading modes, respectively.

The cost associated with offloaded tasks, denoted as F_{pay} , is determined through Equation (21) and (22) as follows:

$$F_{\text{pay}} = \gamma_2 (p_{\text{pay}} + E_{\text{local}})^\gamma, \quad (21)$$

$$F_{\text{pay}} = \gamma_2 \left(p_{\text{pay}} = \sum_{q=1}^m f_{p,q} p_{q,p} + \sum_{r=1}^2 f_{p,r} p_{r,p} + E_{\text{local}} \right)^\gamma, \quad (22)$$

With $\gamma > 0, \gamma_2 > 0, 0 < P_{\min} \leq p_{q,p} \leq P_{\max}$ where E_{local} signifies the electricity consumption associated with local computing by service vehicles and RSUs. p_{pay} represents the total expenses involved in procuring the necessary computing resources essential for task offloading. In this equation, γ signifies the discount factor applied to service agents, while γ_2 represents the discount factor applied to expenses, and $f_{p,q}$ and $f_{p,r}$ denote the computational resources allocated for sub-task processing, acquired from service vehicles and service RSUs, respectively. On the other hand, $p_{q,p}$ and $p_{r,p}$ refer to the unit cost of computational resources supplied by service vehicles and RSUs, respectively.

2) *From Service Agents Perspective:* Service agents, such as RSUs and vehicles, aim to maximize their profit by carrying out the tasks assigned to them.

This profit is calculated by subtracting the task vehicle's computational cost from the costs associated with local computation and cloud server resource procurement. To maintain a positive revenue F_{service} , the price of local resources P_{service} must exceed the cost of cloud resources (P_{cloud}), assuming infinite cloud server resources. The utility function of service agents is

formulated as Equation (23).

$$\max F_{service} = F_{income} - F_{El} - F_{Pcloud}, \quad (23)$$

where F_{income} signifies the revenue generated from task computation, F_{El} stands for the electricity cost associated with local computation, and F_{Pcloud} represents the expense incurred in procuring cloud server resources.

The revenue generated from task computation is defined as:

$$F_{income} = \sum_{p=1}^n (f_{p,q} p_{q,p} + f_{p,r} p_{r,p}), \quad (24)$$

here, $p_{q,p}$ denotes the unit price of computing resources provided by the service vehicle, while $p_{r,p}$ represents the unit price of computing resources offered by the RSU.

The computational electricity cost, denoted as F_{El} , is defined as follows:

$$F_{El} = ek \left(\sum_{p=1}^n (f_{p,q})^2 \frac{I_p}{f_p^{total}} + \sum_{r=1}^2 (f_{p,r})^2 \frac{I_p}{f_p^{total}} \right) \quad (25)$$

where the variable e represents the unit price of electricity. We denoted the power of the equipment as P_{cpu} in alignment with the methodology outlined in [64]. Specifically, $P_{cpu} = kf^2$ is computed as kf^2 , with k signifies the CPU energy coefficient.

Concerning the cost linked to the procurement of resources from the cloud server, denoted as F_{Pcloud} , we have:

$$F_{Pcloud} = P_{cloud} \left(\sum_{q=1}^m f_q^{cloud} + \sum_{r=1}^2 f_r^{cloud} \right) \quad (26)$$

where P_{cloud} represents the unit price of computational resources provided by the cloud server.

IV. PROBLEM SOLUTION

Based on the problem presented in Section III, in this section we analyze the existence of nash equilibrium and propose the reinforcement learning (RL) algorithm combined with nash equilibrium.

The Nash equilibrium concept in game theory promotes optimal resource allocation, giving priority to cooperation over intense competition. Therefore, in the context of the multi-user computer offloading game, Nash equilibrium can mitigate interference resulting from channel competition, favoring a stable state in which each user is mutually satisfied. To explore this concept further, we introduce a Nash equilibrium lemma [66].

Lemma 1: Every finite potential game admits at least one pure strategy nash equilibrium.

In the game, both the number of players and the set of possible strategies for each player are finite. As a result, based on Lemma 1, it can be concluded that there exists at least one pure strategy Nash equilibrium in the multi-agents task offloading game.

Reinforcement Learning (RL) 's essence is to addresses the challenge of decision-making. It comprises three fundamental components: state, action, and reward function. In the context of the multi-user computation offloading system, these elements will be defined in each type of agents.

A. The Leader Agent

In this scenario, we have established the operational framework of the service agent, specifying its state space designated by S_{leader} , its action space designated by A_{leader} and its reward function designated by R_{leader} . These definitions are essential to facilitate efficient decision-making processes.

The state space S_{leader} comprises a diverse set of parameters, including dynamic factors such as:

- the real-time unit price of the cloud server $P_{cloud}(t)$,
- the available computing resources $f^{free}(t)$,
- and the resource utilization $u(t)$ of each service entity,

All contextualized in the communication mode chosen at time t .

- In addition, it integrates essential vehicle data, such as position, speed, direction and acceleration in the context of V2V mode, or the position and coverage area of the RSU service unit in V2I mode.

In this operational framework, **the action space** A_{leader} is uniquely defined for the leader agent and is represented by $p(t)$, signifying the unit price of computing power at time t . This parameter plays a central role in the decision-making process, shaping the agent's strategies and interactions.

To evaluate the effectiveness of these decisions, the reward function R_{leader} is precisely calculated. It is based on the service utility $F_{service}$ and serves as a quantitative measure, effectively evaluating the success of the leader agent's efforts. This reward function is fundamental in guiding the agent's future actions, enabling a continuous process of optimization and adaptation within the operational environment.

B. The Follower Agent

The state space $S_{follower}$ of the task agent covers various aspects, including:

- the decisions of the leader agent, denoted by $P_{service}(t)$,
- the characteristics of parallel tasks such as task size, last execution time, time factor, price factor represented by $T(t)$,
- and the data transmission rates for each link V2V and V2I at time t , denoted by $R(t)$.

The action space $A_{follower}$ available to the follower agent is denoted as $f(t)$, signifying all the computing resources that the task vehicle procures from the service vehicles or RSUs.

The reward function $R_{follower}$ serves as a performance metric for the follower agent, and it is calculated based on the task utility F_{task} . This reward function quantifies the level of success achieved by the follower agent in the given scenario.

According to these three elements, we next propose the MAD-DGP algorithm based on the aforementioned game theory.

C. MADDGP Approach

The application of traditional reinforcement learning to a multi-agent environment is difficult due to the constant learning and changing strategies of each agent. This situation is a source of instability, as changes in one agent's strategy affect another agent's environment. In addition, the probability of a single

agent's state transition can vary from situation to situation, making it difficult to use the experience replay method for learning [68]. The complexity of the environment also increases with the number of agents, making sampling-based gradient estimation optimization methods ineffective. These challenges stem from a lack of interaction between agents, leading to ensemble neglect. To solve these problems, the paper proposes to use the MADDPG algorithm to solve resource purchase problems to compute tasks offloaded to service agents and price service selection for service agents in a multi-agent environment.

The MADDPG algorithm is an extension of the DDPG algorithm that addresses the problem of perception between multiple agents. During training, the actor in the DDPG algorithm selects an action based on the current state s_t , and the critic calculates the Q-value using the state-action function as feedback for the actor. The network parameters are updated based on the difference between the estimated and actual Q-values, and the actor improves its strategies based on the critic's feedback. In the MADDPG algorithm, the critic is able to obtain the state and action of other agents during training, allowing for more accurate Q-value calculations. This means that each agent takes into account not only its own state, but also the behavior of other agents to assess the value of their actions. However, after training, each agent's actor only needs to consider its own state when taking actions, eliminating the need for information from other agents and enabling decentralized execution.

In this article, we consider $\Phi = (\Phi_1, \dots, \Phi_k)$ to denote the strategy parameters of k agents, where $\pi = (\pi_1, \dots, \pi_k)$ represents the corresponding strategy. The formula for the strategy gradient of the i -th agent is given by:

$$\nabla_{\phi_i} J(\phi_i) = \mathbb{E}_{s \sim \rho^\pi, a_i \sim \pi_i} [\nabla_{\phi_i} \log \pi_i(a_i | s_i) \cdot Q_i^\pi(x, a_1, \dots, a_k)] \quad (27)$$

where s_i represents the observation value of the i -th agent, $x = (s_1, \dots, s_k)$ represents the state vector that contains the observation values of all agents, and $Q_i^\pi(x, a_1, \dots, a_k)$ represents the Q-value evaluated by Centralized Critic for the i -th agent. As each agent learns a different Q_i^π function, they may have different reward values for completing cooperative or competitive tasks. For the deterministic strategy μ_i of agent competitive tasks. For the μ_{ϕ_i} , the gradient formula is as follows:

$$\begin{aligned} \nabla_{\phi_i} J(\mu_i) &= E_{x, a \sim D} [\nabla_{\phi_i} \mu_i(a_i | s_i) \\ &\quad \times \nabla_{a_i} Q_i^\mu(x, a_1, \dots, a_k) |_{a_i = \mu_i(s_i)}]. \end{aligned} \quad (28)$$

The experience replay buffer D comprises elements $(x, a_1, \dots, a_k, r_1, \dots, r_k, x')$, storing observation values, actions, and rewards at the current moment, as well as observation values at the next moment for all agents. The update formula for the centralized Critic's action value function Q_i^μ is given by:

$$\begin{cases} L(\phi_i) = \mathbb{E}_{x, a, r, x'} [(Q_i^\mu(x, a_1, \dots, a_k) - y)^2], \\ y = r_i + \gamma Q_i^{\bar{\mu}}(x', a'_1, \dots, a'_k) |_{a'_j = \bar{\mu}_j(s_j)} \end{cases} \quad (29)$$

Here, $\bar{Q}_i^\mu$ represents the target network, and $\bar{\mu} = (\bar{\mu}_{\phi_1}, \dots, \bar{\mu}_{\phi_k})$ denotes the set of target strategies with delay parameters $\bar{\phi}_i$.

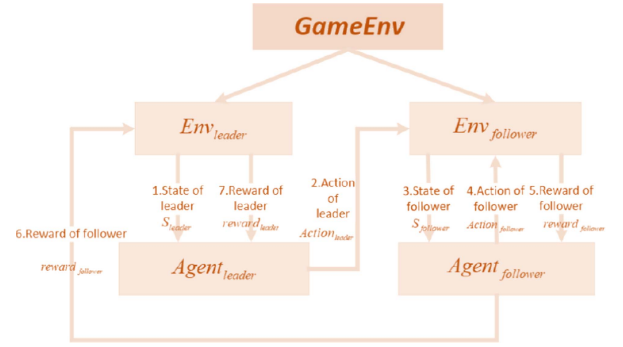


Fig. 3. Agents interaction model.

D. The Interaction Model of the Different Agents

The interaction model between the different agents and their environments according to the Stackelberg game shown in Fig. 3.

This model allows us to distinguish between two types of environments within our simulation environment for our agents. The first type is the environment for the leading agents. These agents initially interact with the simulation environment by observing their current state. Based on this perception, they make pricing decisions that influence the environment for the following agents. Afterward, they receive a positive or negative reward that evaluates the effectiveness of their actions in a specific state. This reward then affects the behavior of the agents in subsequent simulation iterations, guiding them toward the desired outcome.

On the other hand, the follower agents operate within an environment that is influenced by the actions of the leading agents. The perceived states of the follower agents are determined by the pricing strategies employed by the leaders. Once a state is perceived, the follower agent selects a computational task offloading strategy based on the various pricing strategies of the leading agents. Following the implementation of their chosen action, the follower agents are rewarded based on the quality of their performance. This reward further shapes the behavior of the follower agents in future simulation iterations, helping them approach the desired outcome.

Algorithm 1 demonstrates the pseudocode for the 4-Step-Stackelberg-MADDPG algorithm.

V. SIMULATION AND ANALYSIS OF THE SYSTEM

To evaluate the proposed algorithm's performance, we compare it with the following benchmark algorithms as follows.

- MADQN which uses several agents, each equipped with a deep Q network (DQN), to generate policies independently. Each agent interacts with its environment, observes states and takes actions based on the Q values it has learned, with the aim of maximizing its own cumulative reward. [71]
- MAA2C which is an extension of the Advantage Actor-Critic (A2C) algorithm to the multi-agent framework. Each agent has its own networks of actors and critics, who learn to estimate the policy (actor) and value function (critic) independently. Agents collaborate to optimize their

Algorithm 1: 4-Step-Stackelberg-MADDPG Algorithm in the Internet of Vehicles.

Input: Set of services agents vehicles ($v_1, v_2, \dots, v_{n+m}, \dots$) and roadside units (rsu_1, rsu_2),
Initial parameters of services vehicles and rsu
 $\langle x_0, y_0, v_0, a, t, f_{cpu}, f_{time}, f_{pay}, i \rangle$,
IoV environment parameters
($road_size, Dr_{v2v}, Dr_{v2i}, B, P_{v2v}, P_{v2i}, N, h, \theta_1, \theta_2$),
number of episode and number of step in each episode.
Output: Total Utility function values in each episode
1: Initialize agents_env (leader and follower agents)
2: Initialize neural network of agent (leader and follower)
3: Initialize memory space of agent (leader and follower)
4: **for** $episode = 1$ to $n_episode$
5: **for** $step$ to n_step
6: Calculates Data rates in V2V and V2I mode according to equations (2) and (4).
7: Cloud server chooses randomly prices and broadcasts to service agents.
8: Get the states of agents.
9: Obtain the actions of the agents after adding noise to them.
10: Get the agent follower reward.
11: Get the agent leader reward.
12: Record the learning process of the agents.
13: Learn based on MADDPG algorithm of the agents.
14: Update network parameters of agents according to MADDPG algorithm.
15: Go to the next_state of agents
16: **endfor**
17: **endfor**

policies and value functions on the basis of their collective experiences. [72]

- MADDPG, which extends the Deep Deterministic Policy Gradient (DDPG) algorithm to multi-agent scenarios. Each agent maintains its own network of critical actors and learns to optimize its policy on the basis of the centralized value function. MADDPG uses a centralized learning scheme where agents share a common value function but have independent policy networks. [73]

A. Simulation Environment

In our research, we utilized specific technologies and platforms. Python 3.8, PyTorch 1.7, and CUDA 11.0 were employed on the Kaggle platform, utilizing a Compute Engine Python 3 backend processor with 13 GB RAM. Our experiment involved a combination of 5 service vehicles, 24 task vehicles, and 2 RSUs, each equipped with different computing resources. To introduce variability, we randomly generated pricing, delay, and task success sensitivity coefficients, as well as initial coordinates for each vehicle. Table I below describes the parameters used in the experiment.

TABLE I
PARAMETER SETTINGS IN THE SIMULATIONS

Parameter	Value	Parameter	Value
I	[400 ~ 700] MB	f_p^{max}	2.5 Ghz
f_d^{max}	[2 ~ 5] Ghz	v_0	[30 ~ 50] Km/h
P_{V2V}	20 W	P_{V2I}	30 W
B	3.5 Ghz	N	$2, 5 \cdot 10^{-10}$ W/hz
h	4	$F_{success}$	20
P_{cloud}	$1, 5 \cdot 10^{-5}$	u_p	$4 \cdot 10^{-9}$
L	200 m	W	20 m
x	0 ~ 1	y	0 ~ 1
z	0 ~ 1	f_r^{max}	[20 ~ 50] Ghz
w_a	$8 \cdot 10^{-11}$	w_b	$3 \cdot 10^{-9}$
γ_1	10	γ_2	0.15
γ_t	0.25	γ_s	0.0001

B. Deep Reinforcement Learning Model

The parameters of the reinforcement learning algorithm were adjusted through a continuous debugging process.

In our experiment, we used 29 agents, each equipped with four networks: **an actor network, a critical network, a target actor network, and a critical target network.**

- The target networks are copies of the estimated value functions (Q function) of the actor and critical networks. They are kept fixed in order to allow agents to learn from experiments with a fixed policy during a certain number of steps. The use of target networks prevents the learning process from looping and allows the neural network weights to converge to the targets before an update changes the optimization problem.

- Our actor and critical networks are fully connected networks composed of four layers: The first layer is the input layer. For the actor network, the number of neurons corresponds to the dimension of the state vector. For the critical network, the number of neurons is equal to the dimension of the state vector plus the dimension of the action vector. The second layer has 256 neurons. The third layer has 128 neurons. The fourth layer is the output layer. For the actor network, the number of neurons in this layer corresponds to the dimension of the action vector of the agents. The critical network has an output with a single neuron.

We used the Adam optimizer as the training optimizer. For updating the actor and critical networks, we used the stochastic gradient method. The learning rate of the actor network is 0.0001, while that of the critical network is 0.001. We used a reward discount factor of 0.9, a batch size of 64, an experiment playback pool size of 106, and ran 32 training cycles.

C. Results and Analysis

1) *Price Decision for Service Vehicles Agents:* Figs. 4–7 illustrate the evolution of prices for services related to task offloading operations in our system.

First, Fig. 4 shows the pricing policy implemented by the MAA2C algorithm. Prices start high for all vehicles, but fall rapidly over the first 25 episodes, stabilizing around 2 for the majority of vehicles and around 1.5 for vehicle 1 until the end of the episodes. This indicates a convergence of pricing policy with the use of MAA2C.

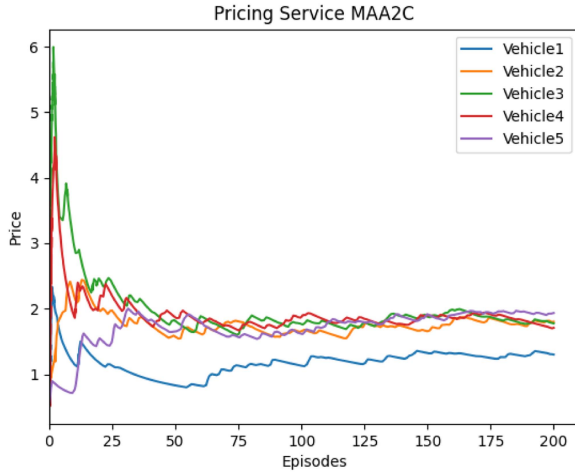


Fig. 4. Price decision of services vehicles with MAA2C.

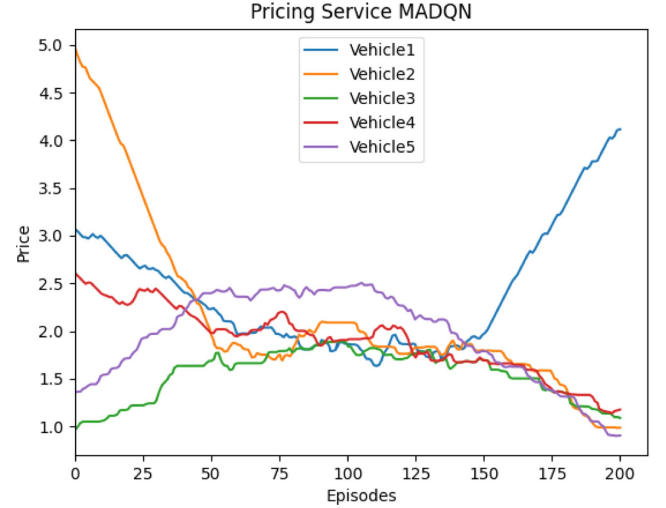


Fig. 6. Price decision of services vehicles with MADQN.

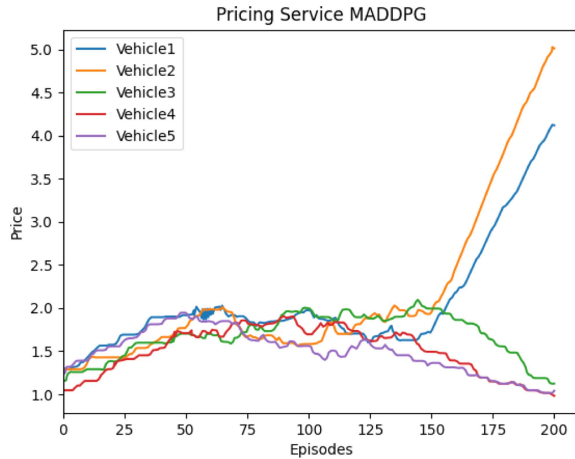


Fig. 5. Price decision of services vehicles with MADDPG.

Next, Fig. 5 shows the results of the MADDPG algorithm. Prices start low for all vehicles, increase slightly, and stabilize for vehicles 3, 4 and 5. However, for vehicles 1 and 2, prices increase significantly after around 125 episodes. This observation suggests that the algorithm is struggling to establish a stable pricing strategy to optimize the utility of service vehicles.

In addition, Fig. 6, using the MADQN algorithm, reveals an initial drop in prices over the first 50 episodes, followed by a period of stabilization between episodes 50 and 125. Towards the final episodes, prices fall for most vehicles, with the exception of vehicle 1, which maintains high prices. As with MADDPG, the algorithm seems to have difficulty establishing a stable pricing policy over the long term.

Finally, Fig. 7, for the 4-Stackelberg-MADDPG algorithm, shows initially very high prices, reaching a maximum of 6, then falling between episodes 25 and 75, before remaining relatively stable until the end of the simulation. This stability indicates a convergence of pricing policy for service vehicles in our scenario.

2) *Average Resource Purchase for Task Vehicles:* Fig. 8 presents the average purchase decisions made by task vehicles

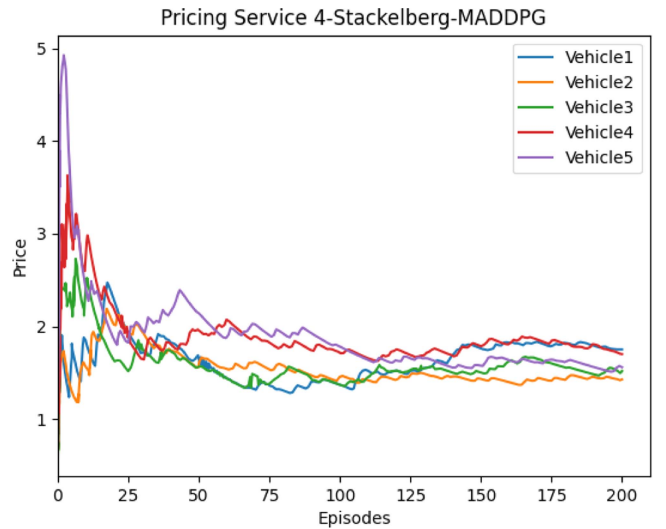


Fig. 7. Price decision of services vehicles with 4-Stackelberg-MADDPG.

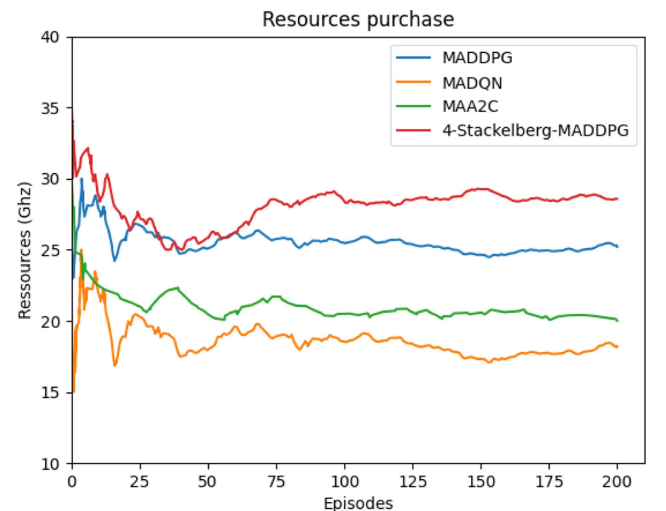


Fig. 8. Average resource purchase to service agents.

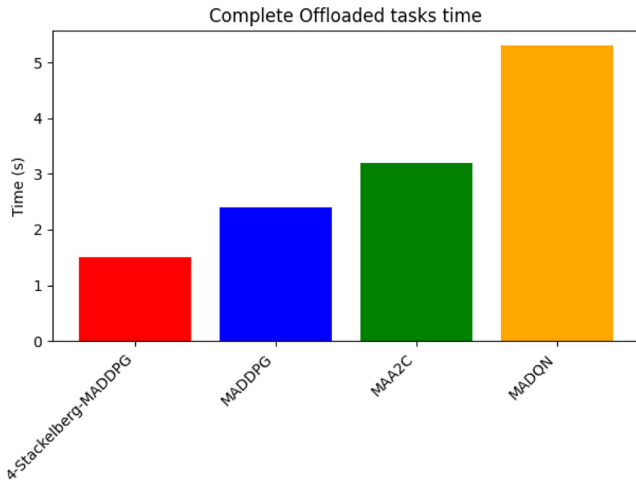


Fig. 9. Time execution of offloaded tasks.

in our simulation. We observe variable amplitude stability for each algorithm.

First, the MAA2C algorithm starts with high resource purchases around 28 Ghz and fluctuates before stabilizing around 21 Ghz. The curve for this algorithm remains relatively stable for the rest of the simulation.

Next, the MADDPG algorithm shows that the amount of resources purchased begins by fluctuating between 25 and 30 Ghz over the first 25 episodes, before stabilizing at a lower average value of 26 Ghz.

Similarly, the MADQN algorithm shows a rapid reduction in resource purchases from 25 Ghz to 17 Ghz, eventually stabilizing around a minimum value of between 17 and 18 Ghz.

Finally, the 4-Stackelberg-MADDPG algorithm shows a rapid drop over the first 50 episodes, from 35 Ghz to 25 Ghz of resources purchased, followed by stabilization at a higher level than MADDPG, around 27 GHz.

3) *Task Probability of Offloaded Task*: After analyzing the resource purchasing strategy for computing offloading operations by vehicles, we observe different performances in terms of task processing time and probability of success in Figs. 9 and 10.

Our solution has a maximum processing time of around 1.5 seconds, suggesting the optimality of the resource purchase choice for processing tasks with the highest performance. In comparison, the MADDPG algorithm has an average processing time of around 2 seconds, reflecting average performance, while the MAA2C algorithm has an average time of around 3 seconds. For its part, the MADQN algorithm requires the longest processing time, around 5 seconds, revealing a lower performance than the other algorithms.

In terms of probability of success, the MADDPG algorithm starts with a high rate around 0.9, declining slightly but stabilizing above 0.8, illustrating consistent performance. MADQN, on the other hand, experienced a more significant decline, stabilizing at around 0.7 despite a similar start to MADDPG, highlighting potential limitations in its computational resource purchasing policy. MAA2C, for its part, shows significant initial variability, but manages to stabilize its probability of success at

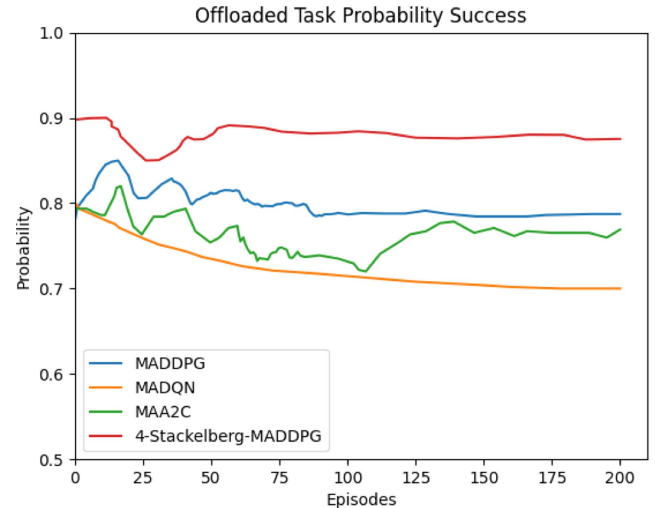


Fig. 10. Task probability success of tasks.

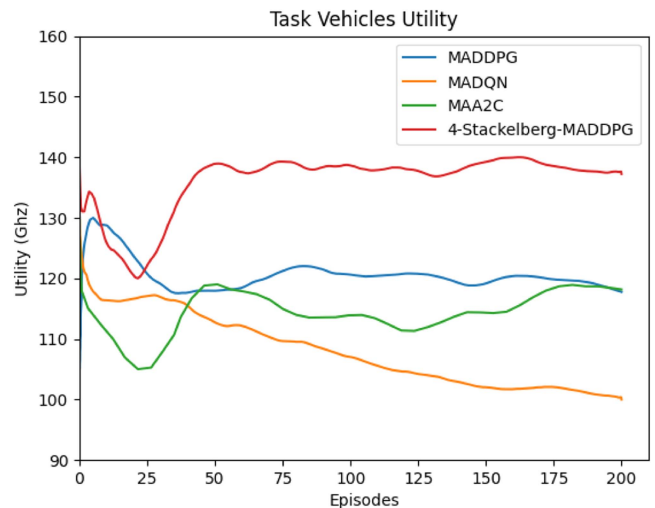


Fig. 11. Change in average utility of task agents.

around 0.78 after the first 50 episodes. Finally, the 4-Stackelberg-MADDPG algorithm stands out with the highest probability of success, initiating slightly at 0.85 before stabilizing at around 0.9, demonstrating superior task success performance.

4) *Average Utility of Agents*: Figs. 11 and 12 illustrate the dynamic evolution of agents' service and task utilities when the unit price is set at $9 \cdot 10^{-9}$. The utility of service vehicles is influenced by various factors, such as the service charge pricing policy and the quantity of resources purchased. For the MADDPG and MADQN algorithms, we observe an increasing utility that approaches maximum efficiency towards the last episodes. However, this utility does not tend to stabilize due to significant variations in vehicle service charges. In contrast, for the MAA2C and 4-Stackelberg-MADDPG algorithms, we observe some fluctuations at the start of the simulation, followed by great stability in the vehicles' utility, which remains around 450.

Similarly, the utility of task agents depends on several factors, including the amount of resources purchased, the time taken to

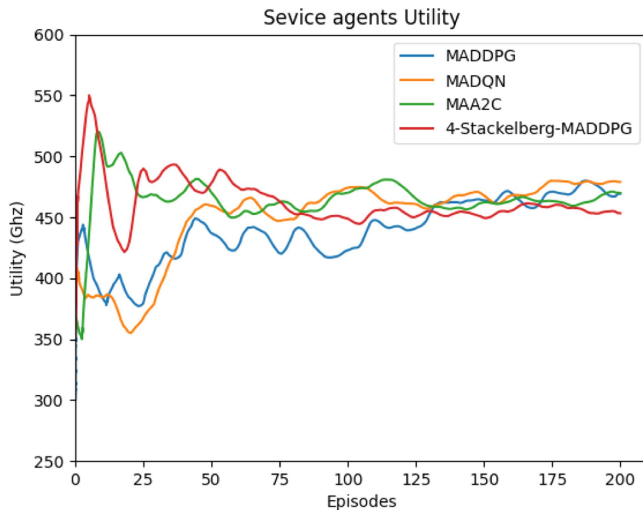


Fig. 12. Change in average utility of services agents.

process tasks and the probability of task success. We observe a stable resource purchase policy with the highest returns for our solution, followed by the MADDPG and MAA2C algorithms. This observation demonstrates effective adaptation of the algorithms to our specific task offloading environment.

However, for the MADQN algorithm, we note a decrease in the utility of task vehicles due to a resource purchasing policy that does not effectively adapt to the conditions of our task offloading environment.

Thus, two Nash equilibria were reached in our analysis. **The MAA2C algorithm leads to a non-optimal Nash equilibrium, characterized by average performance compared with other algorithms due to sub-optimal decisions. In this equilibrium, service prices tend towards 2 for the majority of vehicles, and the quantity of resources purchased stabilizes at around 21 Ghz.**

In contrast, our solution leads to an optimal Nash equilibrium, demonstrating maximum utility performance for service and task agents where service prices remain around 2, and the amount of resources purchased reaches around 27 Ghz. This conclusion suggests that our solution is best suited to this multi-agent task-offloading vehicular scenario.

VI. CONCLUSION

To meet the demands of emerging applications requiring low latency and improved situational awareness, Mobile Edge Computing (MEC) technology has been deployed in vehicular networks. This study introduces MEC technology into vehicular ad hoc networks (VANETs) to establish a vehicular edge computing (VEC) system. Based on the concept of mobile edge computing, we propose a task offloading scheme that promotes collaborative computing between local, edge and cloud resources in the context of the Internet of Vehicles (IoV).

Simulation results demonstrate the performance superiority of our 4- Step-Stackelberg-MADDPG algorithm over other algorithms. Not only does it improve the speed of task execution,

but it also improves task completion success rates. In addition, it effectively balances the utilities of task and service vehicles, as shown by the simulation results.

In the future, it would be interesting to explore the integration of agent reputation into the computational offloading process, particularly in scenarios involving various agent characteristics such as selfishness, greed, or opportunism. By taking an agent's reputation into account, we could potentially influence task offloading decisions, considering the reliability of individual agents. Such integration has the potential to produce more efficient and reliable computation offloading strategies in dynamic environments characterized by diverse agent behaviors.

REFERENCES

- [1] J. Guo, B. Song, Y. He, F. R. Yu, and M. Sookhak, "A survey on compressed sensing in vehicular infotainment systems," *IEEE Commun. Surveys Tut.*, vol. 19, no. 4, pp. 2662–2680, Fourthquarter 2017.
- [2] E. Uhlemann, "Connected-vehicles applications are emerging [connected vehicles]," *IEEE Veh. Technol. Mag.*, vol. 11, no. 1, pp. 25–96, Mar. 2016.
- [3] P. K. Singh, S. K. Nandi, and S. Nandi, "A tutorial survey on vehicular communication state of the art, and future research directions," *Veh. Commun.*, vol. 18, 2019, Art. no. 100164.
- [4] A. Boukerche and V. Soto, "Computation offloading and retrieval for vehicular edge computing: Algorithms, models, and classification," *ACM Comput. Surv.*, vol. 53, no. 4, pp. 1–35, 2021.
- [5] M. A. Khan, "A survey of computation offloading strategies for performance improvement of applications running on mobile devices," *J. Netw. Comput. Appl.*, vol. 56, pp. 28–40, 2015.
- [6] Y. Liu et al., "Dependency-aware task scheduling in vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4961–4971, Jun. 2020.
- [7] F. Arena and G. Pau, "An overview of vehicular communications," *Future Internet*, vol. 11, no. 2, 2019, Art. no. 27.
- [8] J. Wang, C. Jiang, K. Zhang, T. Q. S. Quek, Y. Ren, and L. Hanzo, "Vehicular sensing networks in a smart city: Principles, technologies and applications," *IEEE Wireless Commun.*, vol. 25, no. 1, pp. 122–132, Feb. 2018.
- [9] H. Vahdat-Nejad, A. Ramazani, T. Mohammadi, and W. Mansoor, "A survey on context-aware vehicular network applications," *Veh. Commun.*, vol. 3, pp. 43–57, 2016.
- [10] A. Boukerche and E. Robson, "Vehicular cloud computing: Architectures, applications, and mobility," *Comput. Netw.*, vol. 135, pp. 171–189, 2018.
- [11] K. Zhang, Y. Mao, S. Leng, Y. He, and Y. Zhang, "Mobile-edge computing for vehicular networks: A promising network paradigm with predictive off-loading," *IEEE Veh. Technol. Mag.*, vol. 12, no. 2, pp. 36–44, Jun. 2017.
- [12] H. El-Sayed and M. Chaqfeh, "Exploiting mobile edge computing for enhancing vehicular applications in smart cities," *Sensors*, vol. 19, no. 5, 2019, Art. no. 1073.
- [13] X. Hou et al., "Reliable computation offloading for edge-computingenabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.
- [14] I. A. Abbasi and A. Shahid Khan, "A review of vehicle to vehicle communication protocols for VANETs in the urban environment," *Future Internet*, vol. 10, no. 2, pp. 14–28, 2018.
- [15] C. Silva, L. Silva, L. Santos, J. Sarubbi, and A. Pitsillides, "Broadening understanding on managing the communication infrastructure in vehicular networks: Customizing the coverage using the delta network," *Future Internet*, vol. 11, no. 1, pp. 1–19, 2019.
- [16] C. R. Storck and F. Duarte-Figueiredo, "A survey of 5G technology evolution, standards, and infrastructure associated with vehicle-to-everything communications by Internet of Vehicles," *IEEE Access*, vol. 8, pp. 117593–117614, 2020.
- [17] W. U. Khan, X. Li, A. Ihsan, M. A. Khan, V. G. Menon, and M. Ahmed, "NOMA-enabled optimization framework for nextgeneration small-cell IoV networks under imperfect SIC decoding," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 22442–22451, Nov. 2022, doi: 10.1109/TITS.2021.3091402.
- [18] K. Ganesan, J. Lohr, P. B. Mallick, A. Kunz, and R. Kuchibhotla, "NR sidelink design overview for advanced V2X service," *IEEE Internet Things Mag.*, vol. 3, no. 1, pp. 26–30, Mar. 2020.

- [19] J. Wang, C. Jiang, H. Zhang, Y. Ren, K. -C. Chen, and L. Hanzo, "Thirty years of machine learning: The road to pareto-optimal wireless networks," *IEEE Commun. Surveys Tut.*, vol. 22, no. 3, pp. 1472–1514, Thirdquarter 2020.
- [20] Z. Liu, Z. Li, K. Wu, and M. Li, "Urban traffic prediction from mobility data using deep learning," *IEEE Netw.*, vol. 32, no. 4, pp. 40–46, Jul./Aug. 2018.
- [21] F. Lin et al., "A spatial-temporal hybrid model for short-term traffic prediction," *Math. Problems Eng.*, vol. 2019, 2019, Art. no. 4858546.
- [22] E. Walraven, M. T. Spaan, and B. Bakker, "Traffic flow optimization: A reinforcement learning approach," *Eng. Appl. Artif. Intell.*, vol. 52, pp. 203–212, 2016.
- [23] E. D'Andrea, P. Ducange, B. Lazzerini, and F. Marcelloni, "Real-time detection of traffic from twitter stream analysis," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 4, pp. 2269–2283, Aug. 2015.
- [24] M. Zhu et al., "Safe, efficient, and comfortable velocity control based on reinforcement learning for autonomous driving," *Transp. Res. C: Emerg. Technol.*, vol. 117, 2020, Art. no. 102662.
- [25] Y. Xie, F. Li, Y. Wu, S. Yang, and Y. Wang, "D3-Guard: Acoustic-based drowsy driving detection using smartphones," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2019, pp. 1225–1233.
- [26] J. M. Celaya-Padilla et al., "Texting & driving' detection using deep convolutional neural networks," *Appl. Sci.*, vol. 9, no. 15, 2019, Art. no. 2962.
- [27] H. Ji, O. Alfarrarj, and A. Tolba, "Artificial intelligence-empowered edge of vehicles: Architecture, enabling technologies, and applications," *IEEE Access*, vol. 8, pp. 61020–61034, 2020.
- [28] A. Haydari and Y. Yilmaz, "Deep reinforcement learning for intelligent transportation systems: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 1, pp. 11–32, Jan. 2022.
- [29] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
- [30] J. Liu et al., "RL/DRL meets vehicular task offloading using edge and vehicular cloudlet: A survey," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8315–8338, Jun. 2022.
- [31] L. Xiao, C. Xie, T. Chen, H. Dai, and H. V. Poor, "A mobile offloading game against smart attacks," *IEEE Access*, vol. 4, pp. 2281–2291, 2016.
- [32] Z. Chen and X. Wang, "Decentralized computation offloading for multiuser mobile edge computing: A deep reinforcement learning approach," *EURASIP J. Wireless Commun. Netw.*, vol. 2020, no. 1, p. 188, 2020.
- [33] L. T. Tan and R. Q. Hu, "Mobility-aware edge caching and computing in vehicular networks: A deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 11, pp. 10190–10203, Nov. 2018.
- [34] X. Guo, L. Liu, Z. Chang, and T. Ristaniemi, "Data offloading and task allocation for cloudlet-assisted ad hoc mobile clouds," *Wireless Netw.*, vol. 24, pp. 79–88, 2018.
- [35] L. Song, D. Niyato, Z. Han, and E. Hossain, "Game-theoretic resource allocation methods for device-to-device communication," *IEEE Wireless Commun.*, vol. 21, no. 3, pp. 136–144, Jun. 2014.
- [36] Y. Jie, C. Guo, K. K. R. Choo, and C. Z. Liu, and M. Li, "Game-theoretic resource allocation for fog-based industrial Internet of Things environment," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3041–3052, Apr. 2020.
- [37] F. Li, H. Yao, J. Du, C. Jiang, and Y. Qian, "Stackelberg game-based computation offloading in social and cognitive industrial Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 16, no. 8, pp. 5444–5455, Aug. 2020.
- [38] F. Zeng, Q. Chen, L. Meng, and J. Wu, "Volunteer assisted collaborative offloading and resource allocation in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3247–3257, Jun. 2021.
- [39] F. Zeng, Y. Chen, L. Yao, and J. Wu, "A novel reputation incentive mechanism and game theory analysis for service caching in software-defined vehicle edge com," *Peer-to-Peer Netw. Appl.*, vol. 14, no. 2, pp. 467–481, 2021.
- [40] Z.-L. Chang and H.-Y. Wei, "Flat-rate pricing for green edge computing with latency guarantee: A Stackelberg game approach," in *Proc. IEEE GLOBECOM*, Waikoloa, HI, USA, 2019, pp. 1–6.
- [41] Z. Hong, W. Chen, H. Huang, S. Guo, and Z. Zheng, "Multi-hop cooperative computation offloading for industrial IoT-edge-cloud computing environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 12, pp. 2759–2774, Dec. 2019.
- [42] H. Cao and J. Cai, "Distributed multiuser computation offloading for cloudlet-based mobile cloud computing: A game-theoretic machine learning approach," *IEEE Trans. Veh. Technol.*, vol. 67, no. 1, pp. 752–764, Jan. 2018.
- [43] K. Zhang, X. Gui, D. Ren, and D. Li, "Energy-latency tradeoff for computation offloading in UAV-assisted multiaccess edge computing system," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6709–6719, Apr. 2021.
- [44] H. Zhang, Y. Xiao, S. Bu, D. Niyato, F. R. Yu, and Z. Han, "Computing resource allocation in three-tier IoT fog networks: A joint optimization approach combining Stackelberg game and matching," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1204–1215, Oct. 2017.
- [45] N. Li, J. F. M. Ortega, and V. H. Diaz, "Distributed power control for interference-aware multi-user mobile edge computing: A game theory approach," *IEEE Access*, vol. 6, pp. 36105–36114, 2018.
- [46] X. Ma, C. Lin, X. Xiang, and C. Chen, "Game-theoretic analysis of computation offloading for cloudlet-based mobile cloud computing," in *Proc. 18th ACM Int. Conf.*, 2015, pp. 271–278.
- [47] J. Klaimi, S. M. Senouci, and M. A. Messous, "Theoretical game approach for mobile users resource management in a vehicular fog computing environment," in *Proc. IEEE 14th Int. Wireless Commun. Mobile Comput. Conf.*, 2018, pp. 452–457.
- [48] L. Tang and S. He, "Multi-user computation offloading in MEC: A behavioral perspective," in *Proc. IEEE Netw.*, 2018, pp. 452–457.
- [49] M. Liu and Y. Liu, "Price-based distributed offloading for MEC with computation capacity constraints," *IEEE Wireless Commun. Lett.*, vol. 7, no. 3, pp. 420–423, Jun. 2018.
- [50] L. Long, Z. Liu, Y. Zhou, L. Liu, and J. Shi, "Delay optimized computation offloading and resource allocation for mobile edge computing," in *Proc. IEEE 90th Veh. Technol. Conf.*, 2019, pp. 1–5.
- [51] M. Z. Alam and A. Jamalipour, "Multi-agent DRL-based Hungarian algorithm (MADRLHA) for task offloading in multi-access edge computing Internet of Vehicles (IoVs)," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 7641–7652, Sep. 2022.
- [52] S. Shamsirband et al., "Game theory and evolutionary optimization approaches applied to resource allocation problems in computing environments: A survey," *Math. Biosci. Eng.*, vol. 18, no. 6, pp. 9190–9232, 2021.
- [53] S. Ranadheera, S. Maghsudi, and E. Hossain, "Mobile edge computation offloading using game theory and reinforcement learning," 2017, *arXiv:1711.09012*.
- [54] Y. Zhan, S. Guo, P. Li, and J. Zhang, "A deep reinforcement learning based offloading game in edge computing," *IEEE Trans. Comput.*, vol. 69, no. 6, pp. 883–893, Jun. 2020.
- [55] Wu, Di et al., "The RSU access problem based on evolutionary game theory for VANET," *Int. J. Distrib. Sensor Netw.*, vol. 9, no. 7, 2013, Art. no. 143024.
- [56] C. Wu, X. Chen, T. Yoshinaga, Y. Ji, and Y. Zhang, "Integrating licensed and unlicensed spectrum in the Internet of Vehicles with mobile edge computing," *IEEE Netw.*, vol. 33, no. 4, pp. 48–53, Jul./Aug. 2019, doi: [10.1109/MNET.2019.1800453](https://doi.org/10.1109/MNET.2019.1800453).
- [57] S. Xiao et al., "Research on a task offloading strategy for the internet of vehicles based on reinforcement learning," *Sensors*, vol. 21, no. 18, 2021, Art. no. 6058.
- [58] M. S. Bute, P. Fan, L. Zhang, and F. Abbas, "An efficient distributed task offloading scheme for vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13149–13161, Dec. 2021.
- [59] M. Liwang, J. Wang, Z. Gao, X. Du, and M. Guizani, "Game theory based opportunistic computation offloading in cloud-enabled IoV," *IEEE Access*, vol. 7, pp. 32551–32561, 2019.
- [60] K. Xiong, S. Leng, C. Huang, C. Yuen, and Y. L. Guan, "Intelligent task offloading for heterogeneous V2X communications," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2226–2238, Apr. 2021.
- [61] S. Zhou, Y. Sun, Z. Jiang, and Z. Niu, "Exploiting moving intelligence: Delay-optimized computation offloading in vehicular fog networks," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 49–55, May 2019.
- [62] X. Zhang, M. Peng, S. Yan, and Y. Sun, "Deep-reinforcement-learningbased mode selection and resource allocation for cellular V2X communications," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6380–6391, Jul. 2020.
- [63] M. A. Karabulut, A. F. M. S. Shah, and H. Ilhan, "A novel MIMO-OFDM based MAC protocol for VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20255–20267, Nov. 2022.
- [64] P. Luoto, M. Bennis, P. Pirinen, S. Samarakoon, K. Horneman, and M. Latva-Aho, "Vehicle clustering for improving enhanced LTE-V2X network performance," in *Proc. IEEE Eur. Conf. Netw. Commun.*, 2017, 1–5.
- [65] K. Wang, X. Wang, X. Liu, and A. Jolfaei, "Task offloading strategy based on reinforcement learning computing in edge computing architecture of internet of vehicles," *IEEE Access*, vol. 8, pp. 173779–173789, 2020.
- [66] Q. D. La, Y. H. Chew, and B. H. Soong, "Potential games," in *Potential Game Theory: Applications in Radio Resource Allocation*. Berlin, Germany: Springer, 2016, pp. 23–69.

- [67] W. Fan et al., "Joint task offloading and resource allocation for vehicular edge computing based on V2I and V2V modes," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4277–4292, Apr. 2023.
- [68] N. Fofana, A. B. Letaifa, and A. Rachedi, "Machine learning approach for task offloading strategy in IoV," in *2023 IEEE Int. Wireless Commun. Mobile Comput.*, Marrakesh, Morocco, 2023, pp. 512–517, doi: [10.1109/IWCMC58020.2023.10182970](https://doi.org/10.1109/IWCMC58020.2023.10182970).
- [69] W. Li, B. Jin, and X. Wang, "SparseMAAC: Sparse attention for multi-agent reinforcement learning," in *Database Systems for Advanced Applications*, vol. 11448. Cham, Switzerland: Springer, 2019, pp. 96–110.
- [70] Y. Chen, Z. Li, B. Yang, K. Nai, and K. Li, "A Stackelberg game approach to multiple resources allocation and pricing in mobile edge computing," *Future Gener. Comput. Syst.*, vol. 108, pp. 273–287, 2020.
- [71] B. Yang, L. Zheng, L. J. Ratliff, B. Boots, and J. R. Smith, "Stackelberg MADDPG: Learning emergent behaviors via information asymmetry in competitive games," 2022.
- [72] S. Yao et al., "Blockchain-empowered collaborative task offloading for cloud-edge-device computing," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 12, pp. 3485–3500, Dec. 2022, doi: [10.1109/JSAC.2022.3213358](https://doi.org/10.1109/JSAC.2022.3213358).
- [73] J. Zhang, F. Shen, L. Tang, F. Yan, F. Qin, and C. Wang, "A multi-agent reinforcement learning approach for dynamic offloading with partial information-sharing in IoT networks," in *2023 IEEE 98th Veh. Technol. Conf.*, Hong Kong, 2023, pp. 1–5, doi: [10.1109/VTC2023-Fall60731.2023.10333696](https://doi.org/10.1109/VTC2023-Fall60731.2023.10333696).



Namory Fofana (Member, IEEE) received the graduate degree in Engineering in information security and web technology from the Ecole Supérieure Africaine des TIC, Abidjan, Côte d'Ivoire, in partnership with the Ecole Supérieure de Communication de Tunis (SUP'COM), Aryanah, Tunisia. He is currently working toward the Ph.D. degree in ICT with the Mediatron research Laboratory, SUP'COM, University of Carthage, Carthage, Tunisia. His research interests include reinforcement learning, smart transportation, data processing, and decision-making.



Asma Ben Letaifa (Member, IEEE) received the degree in telecom engineering from the Higher School of Communications (SUP'COM), University of Carthage, Carthage, Tunisia, and the joint Ph.D. degree from SUP'COM, University of Carthage, and from Université de Bretagne Occidentale, Brest, France. She is currently an Associate Professor and a member of the Mediatron Research Laboratory, SUP'COM, University of Carthage. She is the author of several courses on telecommunications services, network modeling with queuing theory, web content, cloud architectures and virtualization, massive big data content, and machine learning algorithms. She is the coauthor of the "Linux practices" MOOC on the FUN platform. She is the author or coauthor of several articles on these subjects. Her research interests include telecom services, cloud and mobile cloud architectures, service orchestration, big data, and quality of experience in an SDN/NFV environment.



Abderrezak Rachedi (Senior Member, IEEE) received the Ph.D. degree in computer science from the University of Avignon, Avignon, France, in 2008, and the Habilitation to Direct Research (HDR) degree from Paris-Est University, Créteil, France, in 2015. He is currently a Full Professor with Université Gustave Eiffel, Paris, France. He has authored or coauthored more than 100 publications in renowned journals and conferences, including IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY and IEEE ICC.

His research interests include wireless networking, IoT, vehicular communication, security, trust models, and blockchain. He has mentored numerous Ph.D. and master's students, served on technical committees, and co-founded the IT startup uGetWin in 2021, focusing on IoT, AI, and digital twins. He serves on editorial boards for IEEE ACCESS, IEEE INTERNET OF THINGS, and other leading journals.